Ponto De Conhecimento Tecnológico

Série 2 — SQL Server

HANDS-ON #07
Índices e Otimização de Consultas

Base de dados: SIManagerDB — Gestão de Serviços e Aplicações

Ferramentas: Microsoft SQL Server, T-SQL, SQL Server Management Studio

Conteúdo: nove secções, medições comparativas, gráficos, planos de execução e validação funcional

Data: novembro de 2026

1. Introdução e Objetivos

1.1. Enquadramento

O desempenho das consultas SQL influencia a eficiência dos sistemas de informação. O crescimento das tabelas pode aumentar leituras, ordenações e operações intermédias. A criação de índices é uma estratégia de otimização, mas os seus benefícios dependem da consulta, dos dados e do plano de execução escolhido.

1.2. Objetivo geral

Analisar o impacto da criação de índices e da reformulação de consultas T-SQL no desempenho das operações de leitura, com cenários de gestão de incidentes.

1.3. Objetivos específicos

• Inventariar os índices existentes e interpretar os respetivos tipos e colunas.

• Analisar planos de execução e operadores relevantes.

• Criar índices simples, compostos e com colunas INCLUDE.

• Medir leituras lógicas, CPU e tempo decorrido.

• Comparar alternativas a funções de janela e validar equivalência funcional.

• Avaliar espaço e estatísticas de utilização dos índices.

1.4. Metodologia

Identificação da consulta → recolha de métricas → análise do plano → alteração controlada → nova medição → comparação → validação funcional. Os resultados correspondem a um ambiente de testes e não constituem garantias para produção.

2. Apresentação da SIManagerDB

A SIManagerDB representa processos de gestão de serviços e aplicações empresariais com dados fictícios. O modelo contém 15 tabelas funcionais.

Domínio	Tabelas
Organização e recursos	Departamento, Equipa, Utilizador, Tecnico
Gestão aplicacional	Aplicacao, ModuloAplicacao, VersaoAplicacao
Fornecedores e licenciamento	Fornecedor, Contrato, Licenca
Gestão de serviços	PoliticaSLA, Incidente, PedidoServico, Intervencao, Alteracao

Indicador	Valor
Aplicações	25
Incidentes	50 000

Indicador	Valor
Incidentes por aplicação	2 000

Dois cenários orientam o estudo: consulta operacional dos incidentes de uma aplicação, ordenados por data; e obtenção dos três incidentes mais recentes por aplicação, acompanhados do respetivo total.

3. Fundamentos de Índices e Planos de Execução

3.1. Tipos de índice

Clustered: organiza os dados segundo a chave do índice. Nonclustered: mantém uma estrutura de acesso separada. Composto: utiliza várias colunas-chave, cuja ordem é relevante. INCLUDE: disponibiliza colunas no nível folha sem as integrar na chave de pesquisa.

3.2. Operadores relevantes

Operador	Significado
Index Scan / Index Seek	Percurso de índice / localização de chaves ou intervalos
Sort	Ordenação explícita
Segment / Sequence Project	Agrupamento de fluxo e cálculo de funções de janela
Stream Aggregate	Agregação de dados
Merge Join	Combinação de fluxos ordenados
Table Spool	Armazenamento temporário de resultados intermédios

Um Index Scan não é necessariamente ineficiente: a análise deve considerar o volume processado e o plano completo.

3.3. Instrumentação

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

As leituras lógicas contabilizam páginas consultadas no buffer pool; as leituras físicas refletem páginas obtidas do armazenamento; CPU e tempo decorrido representam medidas distintas.

4. Exercícios Práticos e Criação de Índices

4.1. Inventário

As vistas sys.indexes, sys.index_columns e sys.columns permitem identificar nomes, tipos, colunas-chave, colunas incluídas e estado dos índices.

4.2. Consulta operacional

```
SELECT IncidenteId, AplicacaoId, Titulo, Estado, DataAbertura
FROM dbo.Incidente
```

```
WHERE AplicacaoId = @AplicacaoId
ORDER BY DataAbertura DESC;
```

A consulta devolve 2 000 incidentes para uma aplicação do conjunto de teste.

4.3. Índice simples

```
CREATE NONCLUSTERED INDEX IX_Incidente_AplicacaoId
ON dbo.Incidente (AplicacaoId);
```

O índice simples suporta o filtro, mas não resolve necessariamente a ordenação e a cobertura das colunas selecionadas.

4.4. Índice composto com INCLUDE

```
CREATE NONCLUSTERED INDEX IX_Incidente_Aplicacao_DataAbertura
ON dbo.Incidente (AplicacaoId ASC, DataAbertura DESC)
INCLUDE (Titulo, Estado);
```

O índice composto alinha filtro e ordenação e disponibiliza as colunas descritivas no nível folha.

4.5. Índice para ROW_NUMBER()

```
CREATE NONCLUSTERED INDEX IX_Incidente_Aplicacao_Data_Incidente
ON dbo.Incidente
(AplicacaoId, DataAbertura DESC, IncidenteId DESC);
```

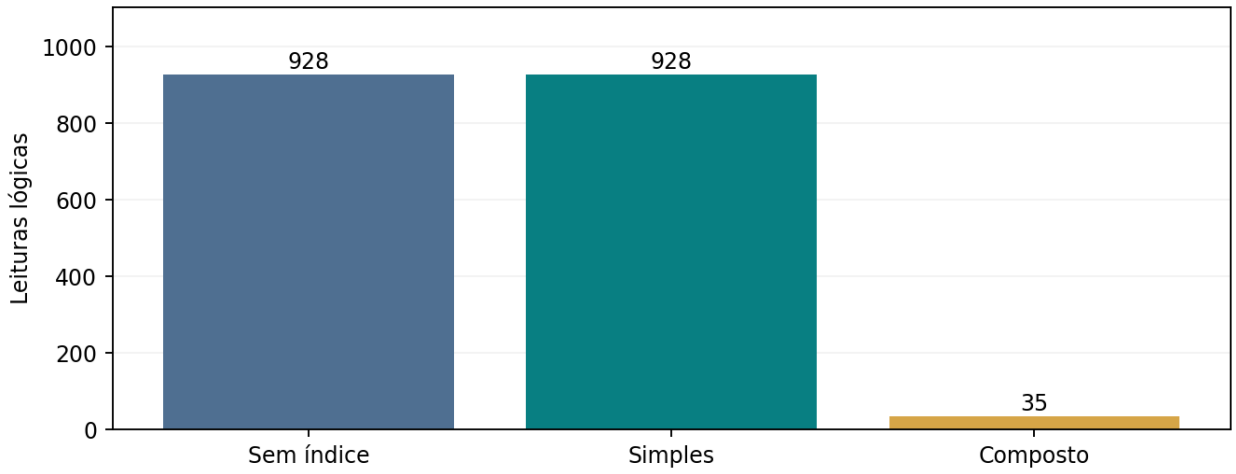
A estrutura corresponde às colunas de PARTITION BY e ORDER BY da função de janela. Os índices compostos foram confirmados como NONCLUSTERED e ativos.

5. Comparação de Desempenho e Gráficos

5.1. Consulta operacional

Cenário	Leituras lógicas	Registos
Sem índice adicional	928	2 000
Índice simples	928	2 000
Índice composto com INCLUDE	35	2 000

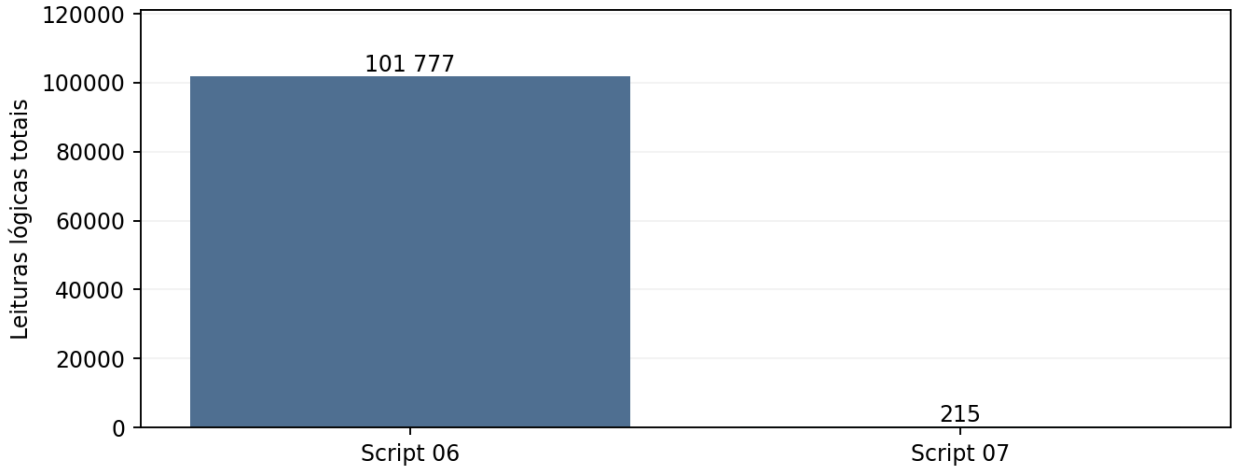
Redução observada: 96,23% das leituras lógicas entre o cenário sem índice adicional e o índice composto. Os valores 928/928/35 são medições históricas registadas no exercício.

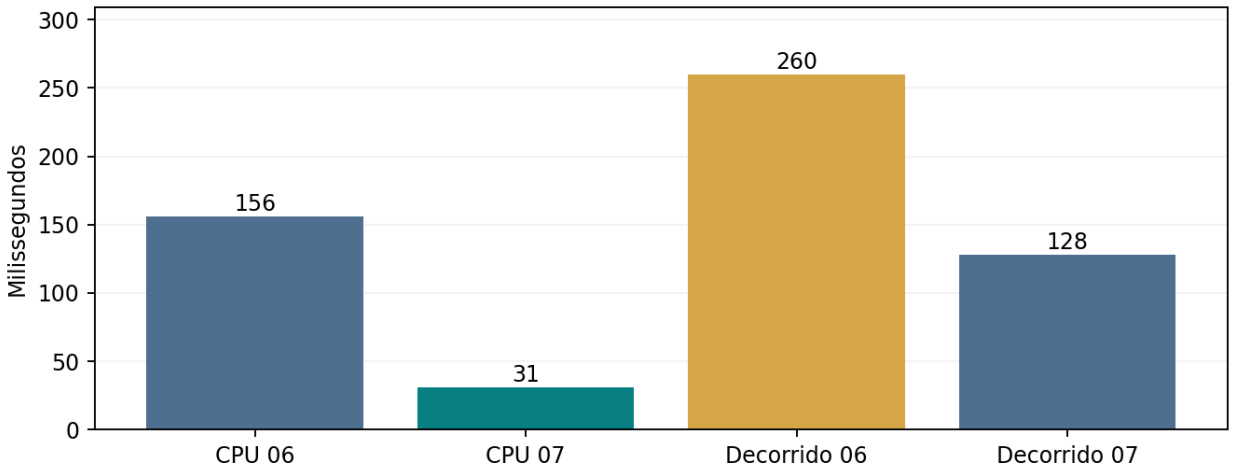


5.2. Scripts 06 e 07

Métrica	Script 06	Script 07
Leituras Incidente	126	215
Leituras Worktable	101 651	0
Leituras totais	101 777	215
CPU	156 ms	31 ms
Tempo decorrido	260 ms	128 ms
Registos devolvidos	75	75

A redução total de leituras lógicas foi de 99,79%; a CPU diminuiu 80,13% e o tempo decorrido 50,77%. Embora as leituras da tabela Incidente tenham aumentado, a eliminação das leituras em Worktable determinou a redução global.





Os tempos são observações de execuções específicas; cache, concorrência, estatísticas e recursos do servidor podem alterar os resultados. Para benchmarking rigoroso, devem repetir-se os ensaios em condições controladas.

6. Otimização das Funções de Janela

6.1. Formulação inicial — Script 06

```
ROW_NUMBER() OVER (
 PARTITION BY i.AplicacaoId
 ORDER BY i.DataAbertura DESC, i.IncidenteId DESC
) AS Posicao,
COUNT(*) OVER (PARTITION BY i.AplicacaoId)
 AS TotalIncidentesAplicacao
```

O plano analisado utilizou operadores Table Spool e produziu 101 651 leituras lógicas em Worktable.

6.2. Reformulação — Script 07

```
WITH CTE_Totais AS (
 SELECT AplicacaoId, COUNT(*) AS TotalIncidentesAplicacao
 FROM dbo.Incidente
 GROUP BY AplicacaoId
)
```

A reformulação separa a agregação dos totais do cálculo de ROW_NUMBER() e combina os resultados através de JOIN. O plano passou a utilizar, entre outros, Stream Aggregate, Segment, Sequence Project e Merge Join, sem os Table Spool observados no plano anterior.

A conclusão é específica deste cenário: GROUP BY + JOIN não é universalmente superior a COUNT() OVER().

7. Validação da Equivalência Funcional

O Script 08 guardou os resultados dos Scripts 06 e 07 em tabelas temporárias, comparou-os com EXCEPT nos dois sentidos e verificou duplicados. Foram confrontadas AplicacaoId, IncidenteId, DataAbertura, Posicao e TotalIncidentesAplicacao.

Indicador	Resultado
Registos Script 06	75
Registos Script 07	75
Diferenças 06 → 07	0
Diferenças 07 → 06	0
Duplicados Script 06	0
Duplicados Script 07	0

VALIDADO — resultados equivalentes (75 registos). A otimização preservou os resultados funcionais no conjunto de dados analisado.

8. Conclusões e Recomendações

Os resultados demonstram que a adequação da chave e da cobertura do índice ao padrão de consulta é determinante. O índice simples não reduziu as leituras da consulta operacional; o índice composto reduziu-as em 96,23%.

A reformulação da consulta com funções de janela reduziu as leituras lógicas totais em 99,79%, com resultados funcionais equivalentes confirmados por comparação integral.

• Analisar os planos de execução antes de criar índices.

• Considerar filtros, ordenações e colunas projetadas em conjunto.

• Evitar índices redundantes e avaliar custos de armazenamento e escrita.

• Medir leituras lógicas, CPU e tempo decorrido, sem depender de uma única execução.

• Validar a correção dos resultados após cada alteração.

• Monitorizar estatísticas de utilização e reavaliar índices à medida que o sistema evolui.

Conclusão: uma consulta otimizada deve ser eficiente, correta e tecnicamente justificável.

9. Anexos — Scripts SQL e Evidências

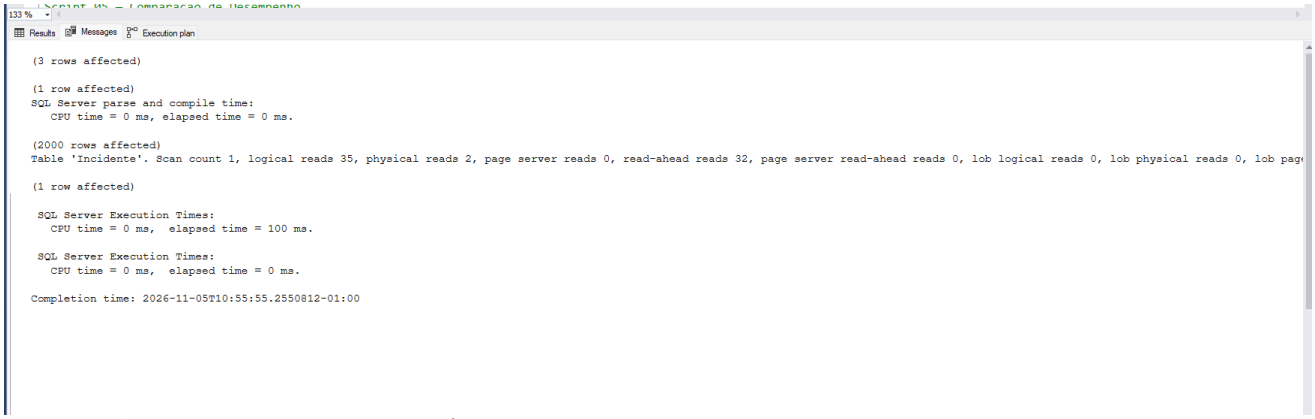
9.1. Organização dos scripts

Script	Objetivo
01 — Inventário	Identificação de índices e respetivas colunas
02 — Consulta baseline	Medição da consulta operacional
03 — Índice simples	Criação e verificação do índice por aplicação
04 — Índice composto	Índice com ordenação e INCLUDE
05 — Comparação	Comparação das leituras históricas
06 — ROW_NUMBER + COUNT OVER	Formulação inicial
07 — Otimização GROUP BY	Formulação alternativa
08 — Validação de equivalência	Comparação integral dos resultados

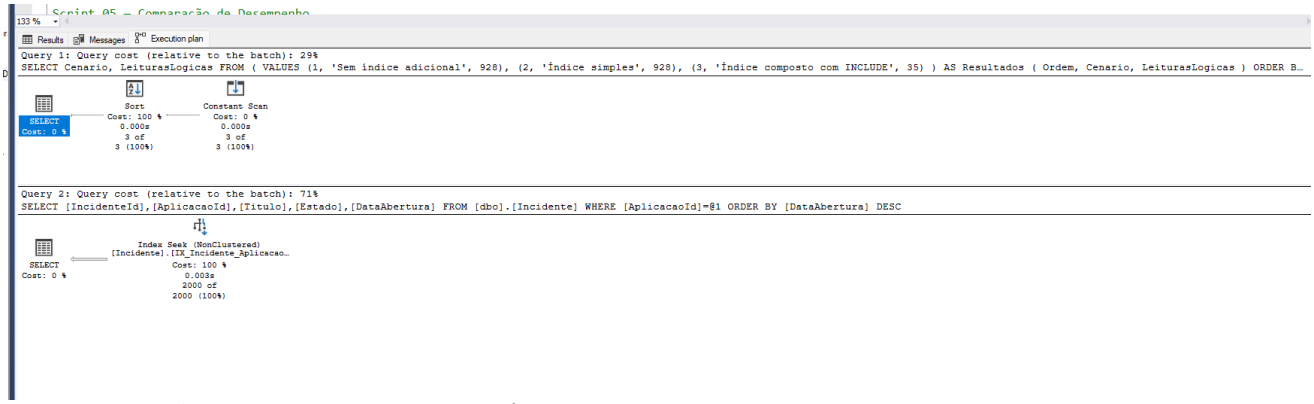
Os nomes descritivos dos primeiros cinco scripts são uma organização proposta. Os Scripts 06, 07 e 08 correspondem às versões de trabalho utilizadas nas medições e validação.

9.2. Capturas do SSMS

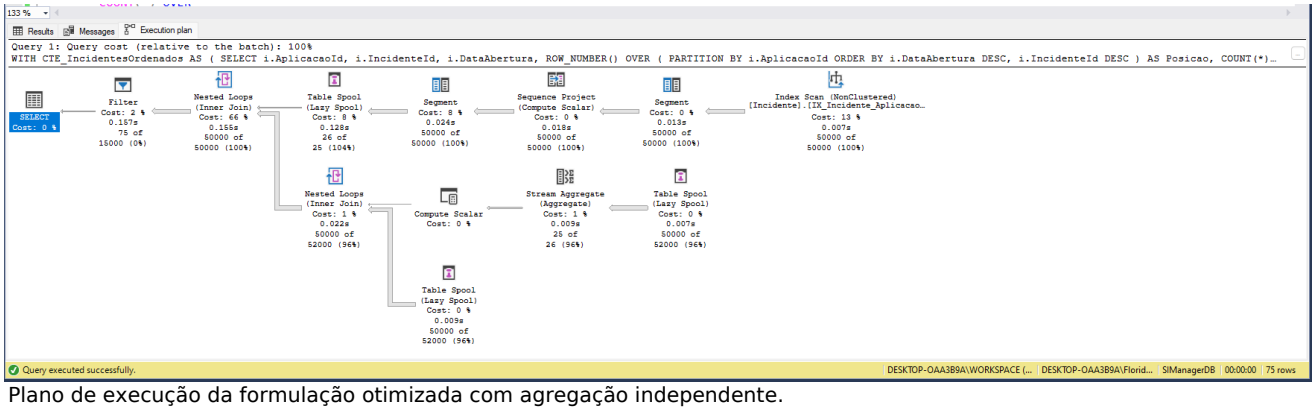
As capturas seguintes documentam resultados e planos de execução disponibilizados durante o desenvolvimento. Foram mantidas como evidência visual; a legibilidade de pormenores depende da ampliação do PDF.



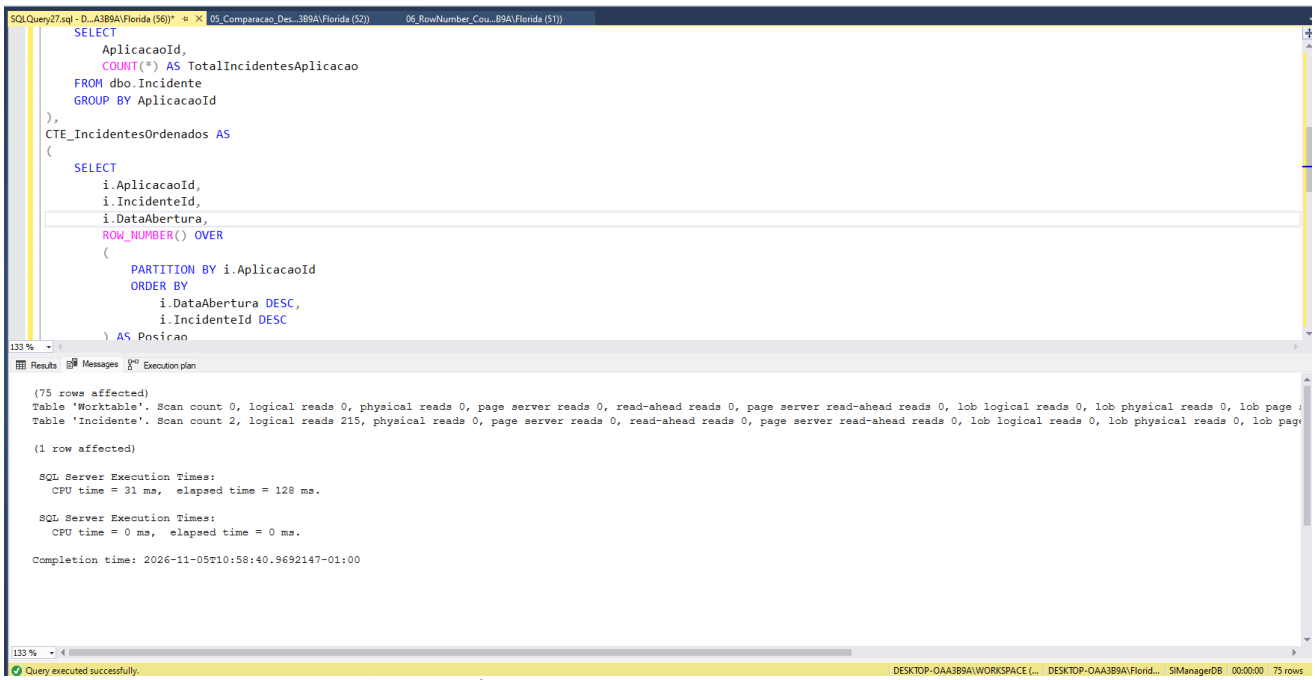
Comparação de desempenho: 35 leituras lógicas na consulta operacional.



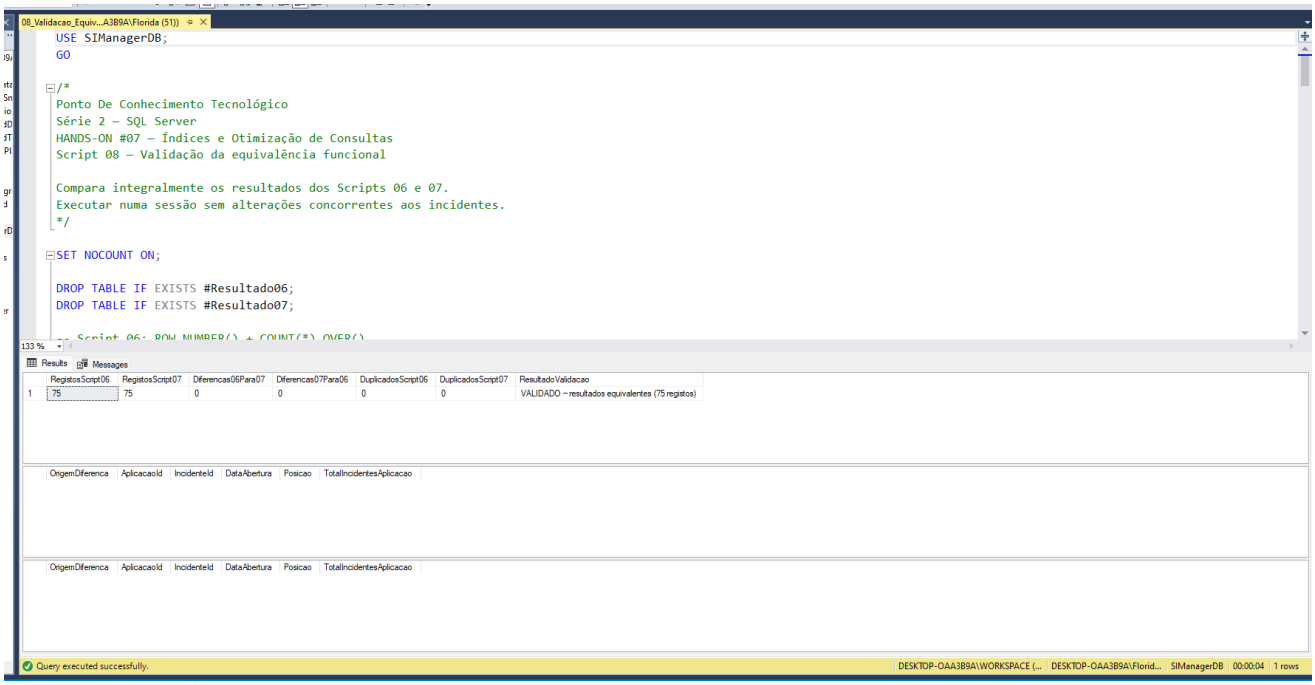
Plano de execução da consulta operacional com índice nonclustered.



Plano de execução da formulação otimizada com agregação independente.



Medições do Script 06 com operações intermédias.



Script 08: validação funcional de 75 registos equivalentes.

9.3. Reprodutibilidade

As métricas refletem o estado da base de dados, os índices e as condições da sessão no momento da recolha. A reprodução exige atenção à ordem de criação dos índices e às condições de execução. As medições históricas não devem ser apresentadas como resultados garantidos em novas execuções.